مهندسی نرم‌افزار

جلسهٔ ۶ — طراحی شیءگرا و SOLID

ابراهیم فضلی

ترم ۱۴۰۴

مرور جلسهٔ ۵

- Review: Sprint نمایش محصول، نه گزارش
- Done of Definition
- Retrospective: Continue / Stop / Start
- درس‌های Sprint ۱
- برنامهٔ Sprint ۲

امروز: چطور کد بهتر بنویسیم؟

چرا طراحی شیءگرا مهم است؟

- کد خوب = قابل نگهداری
- کد بد = هزینهٔ تغییر بالا
- طراحی شیءگرا = مدیریت پیچیدگی
- SOLID = اصول طراحی خوب

۸۰٪ زمان توسعه، صرف خواندن کد می‌شود.

چهار اصل OOP

اصل	معنی
Encapsulation	کپسوله‌سازی داده و رفتار
Inheritance	ارث‌بری از کلاس والد
Polymorphism	چندشکلی — یک نام، چند رفتار
Abstraction	انتزاع — پنهان کردن جزئیات

این چهار اصل، پایهٔ طراحی شیءگرا هستند.

Encapsulation – کپسوله‌سازی

داده و متدها در یک واحد (کلاس) قرار می‌گیرند.

- داده‌های داخلی پنهان هستند
- دسترسی فقط از طریق متدهای عمومی
- مزیت: تغییر داخلی، بدون تأثیر بیرونی

مثال: کلاس Product – قیمت خصوصی، متد getPrice() عمومی

Inheritance — ارث‌بری

کلاس فرزند، ویژگی‌های کلاس والد را به ارث می‌برد.

- مزیت: استفادهٔ مجدد از کد
- عیب: وابستگی زیاد
- توصیه: ترکیب بهتر از ارث‌بری است

مثال: Admin از User ارث می‌برد

Polymorphism – چندشکلی

یک نام، چند رفتار.

- متد مشترک، پیاده‌سازی متفاوت
- مزیت: کد عمومی، رفتار خاص
- مثال: pay() برای کارت، PayPal، درگاه

یک interface، چند implementation

Abstraction — انتزاع

پنهان کردن جزئیات پیاده‌سازی.

- کاربر چه کاری انجام می‌دهد را می‌بیند
- چطور انجام می‌دهد را نمی‌بیند
- ابزار: Class Abstract Interface،

مثال: UserRepository.save() — بدون دانستن جزئیات دیتابیس

SOLID چیست؟

پنج اصل طراحی شیءگرا

حرف	اصل
S	Principle Responsibility Single
O	Principle Open/Closed
L	Principle Substitution Liskov
I	Principle Segregation Interface
D	Principle Inversion Dependency

Responsibility Single – S

هر کلاس فقط یک مسئولیت

- کلاس فقط یک دلیل برای تغییر داشته باشد
- اگر کلاس هم محاسبه، هم ذخیره، هم چاپ کند نقض SRP

مثال: Order نباید فاکتور چاپ کند. Invoice جدا باشد.

Open/Closed – O

باز برای توسعه، بسته برای تغییر

- افزودن ویژگی جدید بدون تغییر کد قدیمی
- ابزار: Class Abstract Interface،

مثال: افزودن روش پرداخت جدید، بدون تغییر کد اصلی

Substitution Liskov — L

فرزند باید جایگزین والد شود

- هر جا User انتظار می‌رود، Admin هم کار کند
- فرزند نباید رفتار والد را نقض کند

مثال بد: Square از Rectangle ارث می‌برد، ولی setWidth() را نقض می‌کند

Segregation Interface — I

رابط‌های کوچک، نه بزرگ

- کلاس نباید متدهای غیرضروری را پیاده کند
- چند interface کوچک بهتر از یک interface بزرگ

مثال: PaymentGateway و RefundGateway جدا باشند

Inversion Dependency — D

به انتزاع وابسته باش، نه پیاده‌سازی

- کلاس سطح بالا نباید به کلاس سطح پایین وابسته باشد
- هر دو به interface وابسته باشند

مثال: OrderService به IOrderRepository وابسته باشد، نه به PostgresOrderRepository

مثال: نقض SRP در HomeStore

کلاس Order که همه‌کاره است:

- محاسبهٔ مجموع
- ذخیره در دیتابیس
- ارسال ایمیل تأیید
- چاپ فاکتور
- اطلاع به انبار

۵ دلیل برای تغییر = نقض شدید SRP

Refactor با SOLID

جدا کردن مسئولیت‌ها:

کلاس	مسئولیت
Order	محاسبهٔ مجموع
OrderRepository	ذخیره در دیتابیس
EmailService	ارسال ایمیل
InvoiceService	چاپ فاکتور
StockService	اطلاع به انبار

هر کلاس = یک مسئولیت

Coupling و Cohesion

مفهوم	توضیح
Coupling	وابستگی بین کلاس‌ها — کم بهتر
Cohesion	ارتباط داخلی کلاس — زیاد بهتر

- Coupling کم: تغییر یک کلاس، بقیه را نشکند
- Cohesion زیاد: همهٔ متدهای کلاس، یک هدف

مثال: Coupling در HomeStore

کد با Coupling بالا:

- OrderService مستقیماً PostgresOrderRepository را می‌سازد
- تغییر دیتابیس = تغییر OrderService

کد با Coupling پایین:

- OrderService به IOrderRepository وابسته است
- تغییر دیتابیس = فقط تغییر implementation

مقدمه — Patterns Design

راه‌حل‌های آماده برای مشکلات رایج

- Creational: Builder Singleton، Factory
- Structural: Facade Decorator، Adapter
- Behavioral: Command Observer، Strategy

در جلسهٔ ۷ عمیق‌تر می‌بینیم.

تمرین: Refactor کد

کد زیر را با SOLID بازنویسی کنید:

کلاس	مسئولیت‌ها
ProductManager	افزودن، ویرایش، حذف، جست‌وجو، گزارش
OrderManager	ثبت، لغو، پیگیری، ایمیل، فاکتور

سؤال: چند کلاس جدید لازم است؟ هر کلاس چه مسئولیتی دارد؟

تسک هفتهٔ آینده

تسک شما:

- کد Sprint ۱ را بازبینی کنید
- کلاس‌های بزرگ را جدا کنید
- SRP را رعایت کنید
- Coupling را کم کنید
- مستندسازی تصمیم‌ها

تسک من:

- Refactor کد HomeStore
- مستند SOLID
- جزوهٔ مطالعه
- آماده‌سازی جلسهٔ ۷

سه برداشت کلیدی

۱. SOLID = پنج اصل برای کد قابل نگهداری
۲. Coupling کم، Cohesion زیاد
۳. Refactor یک عادت است، نه یک پروژه

سؤالی دارید؟ به امید یک ترم پربار